

Практическая работа 3: "Система управления умным домом"

Цель работы

Разработать консольное приложение для управления устройствами умного дома, демонстрирующее применение классических паттернов проектирования Gang of Four.

Описание системы

Система должна моделировать умный дом со следующими возможностями:

1. **Управление устройствами:** включение/выключение различных устройств
2. **Создание устройств:** фабрики для создания разных типов устройств
3. **Расширение функциональности:** добавление возможностей к устройствам
4. **Уведомления:** система оповещения об изменениях в доме
5. **История команд:** возможность отмены выполненных действий

Основные сущности системы

- **Устройства:** лампы, термостаты, камеры безопасности, умные розетки
- **Команды:** действия над устройствами
- **Наблюдатели:** подписчики на события в доме
- **Адаптеры:** интеграция старых устройств

Техническое задание

Часть 1: Порождающие паттерны

1.1 Singleton - Центр управления домом

Создайте класс `SmartHomeController`, который должен быть единственным в системе (Singleton).

Требования:

- Приватный конструктор
- Статический метод `getInstance()` для получения экземпляра
- Потокобезопасная реализация (Double-Checked Locking)

- Список всех устройств в доме
- Методы для добавления/удаления устройств

Заготовка кода:

```
public class SmartHomeController {
    private static volatile SmartHomeController instance;
    private List<Device> devices;

    private SmartHomeController() {
        // TODO: Инициализировать список устройств
        System.out.println("Система умного дома инициализирована");
    }

    public static SmartHomeController getInstance() {
        // TODO: Реализовать Double-Checked Locking
        // Проверка instance == null
        // Синхронизация
        // Повторная проверка
    }

    public void registerDevice(Device device) {
        // TODO: Добавить устройство в список
    }

    public void listDevices() {
        // TODO: Вывести все устройства
    }
}
```

1.2 Factory Method - Фабрика устройств

Создайте иерархию фабрик для создания различных устройств.

Требования:

- Интерфейс Device с методами turnOn(), turnOff(), getStatus()
- Абстрактный класс DeviceFactory с фабричным методом
- Конкретные фабрики: LightFactory, ThermostatFactory, CameraFactory

Заготовка кода:

```
// Интерфейс устройства
public interface Device {
    void turnOn();
    void turnOff();
    String getStatus();
    String getName();
}
```

```
// Абстрактная фабрика
public abstract class DeviceFactory {
    // Фабричный метод
    public abstract Device createDevice(String name);

    // Шаблонный метод для создания и регистрации
    public Device createAndRegister(String name) {
        Device device = createDevice(name);
        SmartHomeController.getInstance().registerDevice(device);
        System.out.println("Устройство " + name + " создано и зарегистрировано");
        return device;
    }
}

// Конкретная фабрика для ламп
public class LightFactory extends DeviceFactory {
    @Override
    public Device createDevice(String name) {
        // TODO: Вернуть новый объект Light
    }
}

// Класс Light
public class Light implements Device {
    private String name;
    private boolean isOn;

    public Light(String name) {
        // TODO: Инициализировать поля
    }

    @Override
    public void turnOn() {
        // TODO: Включить свет
    }

    @Override
    public void turnOff() {
        // TODO: Выключить свет
    }

    @Override
    public String getStatus() {
        // TODO: Вернуть статус (включен/выключен)
    }

    @Override
    public String getName() {
        return name;
    }
}
```

Задание:

1. Реализуйте классы `Thermostat` и `Camera` аналогично `Light`
2. Создайте фабрики `ThermostatFactory` и `CameraFactory`
3. Термостат должен иметь дополнительное поле `temperature` и метод `setTemperature(int temp)`
4. Камера должна иметь метод `startRecording()` и `stopRecording()`

1.3 Builder - Конфигурация умного дома

Создайте Builder для пошагового создания конфигурации умного дома.

Заготовка кода:

```
public class SmartHomeConfiguration {
    private final String homeName;
    private final String ownerName;
    private final int roomCount;
    private final boolean hasSecuritySystem;
    private final boolean hasClimateControl;
    private final String wifiSSID;

    private SmartHomeConfiguration(Builder builder) {
        // TODO: Инициализировать все поля из builder
    }

    public static class Builder {
        // Обязательные параметры
        private final String homeName;
        private final String ownerName;

        // Опциональные параметры с значениями по умолчанию
        private int roomCount = 1;
        private boolean hasSecuritySystem = false;
        private boolean hasClimateControl = false;
        private String wifiSSID = "SmartHome_Default";

        public Builder(String homeName, String ownerName) {
            // TODO: Инициализировать обязательные параметры
        }

        public Builder roomCount(int count) {
            // TODO: Установить количество комнат и вернуть this
        }

        public Builder withSecuritySystem() {
            // TODO: Включить систему безопасности и вернуть this
        }

        public Builder withClimateControl() {
```

```

        // TODO: Включить климат-контроль и вернуть this
    }

    public Builder wifiSSID(String ssid) {
        // TODO: Установить WiFi SSID и вернуть this
    }

    public SmartHomeConfiguration build() {
        // TODO: Вернуть новый объект SmartHomeConfiguration
    }
}

@Override
public String toString() {
    // TODO: Вернуть строковое представление конфигурации
}
}

```

Часть 2: Структурные паттерны

2.1 Adapter - Адаптация старых устройств

Создайте адаптер для интеграции старых устройств в систему умного дома.

Сценарий: У вас есть старая лампа с методами `switchOn()` и `switchOff()`, но ваша система ожидает методы `turnOn()` и `turnOff()`.

Заготовка кода:

```

// Старое устройство (несовместимый интерфейс)
public class LegacyLamp {
    private String name;
    private boolean powered;

    public LegacyLamp(String name) {
        this.name = name;
        this.powered = false;
    }

    public void switchOn() {
        powered = true;
        System.out.println("Старая лампа " + name + " включена");
    }

    public void switchOff() {
        powered = false;
        System.out.println("Старая лампа " + name + " выключена");
    }
}

```

```

    public boolean isPowered() {
        return powered;
    }

    public String getLampName() {
        return name;
    }
}

// Адаптер
public class LegacyLampAdapter implements Device {
    private LegacyLamp legacyLamp;

    public LegacyLampAdapter(LegacyLamp legacyLamp) {
        // TODO: Сохранить ссылку на старую лампу
    }

    @Override
    public void turnOn() {
        // TODO: Вызвать switchOn() у legacyLamp
    }

    @Override
    public void turnOff() {
        // TODO: Вызвать switchOff() у legacyLamp
    }

    @Override
    public String getStatus() {
        // TODO: Вернуть статус на основе isPowered()
    }

    @Override
    public String getName() {
        // TODO: Вернуть имя из getLampName()
    }
}

```

2.2 Decorator - Расширение функциональности устройств

Создайте декораторы для динамического добавления возможностей устройствам.

Сценарий: Добавьте возможность логирования действий и энергосбережения к любому устройству.

Заготовка кода:

```

// Базовый декоратор
public abstract class DeviceDecorator implements Device {
    protected Device decoratedDevice;
}

```

```

public DeviceDecorator(Device device) {
    // TODO: Сохранить ссылку на декорируемое устройство
}

@Override
public void turnOn() {
    // TODO: Делегировать вызов декорируемому устройству
}

@Override
public void turnOff() {
    // TODO: Делегировать вызов декорируемому устройству
}

@Override
public String getStatus() {
    // TODO: Делегировать вызов декорируемому устройству
}

@Override
public String getName() {
    // TODO: Делегировать вызов декорируемому устройству
}
}

// Декоратор для логирования
public class LoggingDecorator extends DeviceDecorator {
    public LoggingDecorator(Device device) {
        super(device);
    }

    @Override
    public void turnOn() {
        System.out.println("[LOG] Включение устройства: " + getName());
        System.out.println("[LOG] Время: " + java.time.LocalDateTime.now());
        // TODO: Вызвать метод родителя
    }

    @Override
    public void turnOff() {
        System.out.println("[LOG] Выключение устройства: " + getName());
        System.out.println("[LOG] Время: " + java.time.LocalDateTime.now());
        // TODO: Вызвать метод родителя
    }
}

// Декоратор для энергосбережения
public class EnergySavingDecorator extends DeviceDecorator {
    private int energySaved;

    public EnergySavingDecorator(Device device) {
        super(device);
        this.energySaved = 0;
    }
}

```

```

    }

    @Override
    public void turnOn() {
        System.out.println("[ENERGY] Режим энергосбережения активен для " + getName());
        // TODO: Вызвать метод родителя
        energySaved += 10; // условные единицы
    }

    @Override
    public String getStatus() {
        // TODO: Вернуть статус + информацию об энергосбережении
    }

    public int getEnergySaved() {
        return energySaved;
    }
}

```

Задание: Создайте декоратор `SecurityDecorator` , который:

- При включении устройства проверяет пароль
- Логирует все попытки доступа
- Блокирует устройство после 3 неудачных попыток

2.3 Facade - Упрощённый интерфейс

Создайте фасад для упрощения управления всей системой умного дома.

Заготовка кода:

```

public class SmartHomeFacade {
    private SmartHomeController controller;
    private LightFactory lightFactory;
    private ThermostatFactory thermostatFactory;
    private CameraFactory cameraFactory;

    public SmartHomeFacade() {
        // TODO: Инициализировать контроллер и фабрики
    }

    // Упрощённые методы для типичных сценариев
    public void morningRoutine() {
        System.out.println("=== Запуск утреннего режима ===");
        // TODO:
        // 1. Включить все лампы
        // 2. Установить температуру 22 градуса
        // 3. Выключить камеры
    }
}

```



```

public void eveningRoutine() {
    System.out.println("=== Запуск вечернего режима ===");
    // TODO:
    // 1. Приглушить свет
    // 2. Установить температуру 20 градусов
    // 3. Включить камеры
}

public void leaveHome() {
    System.out.println("=== Режим 'Ухожу из дома' ===");
    // TODO:
    // 1. Выключить все лампы
    // 2. Установить температуру 18 градусов
    // 3. Включить все камеры
    // 4. Включить систему безопасности
}

public void arriveHome() {
    System.out.println("=== Режим 'Возвращение домой' ===");
    // TODO:
    // 1. Включить свет в прихожей
    // 2. Установить комфортную температуру
    // 3. Выключить систему безопасности
}
}

```

Часть 3: Поведенческие паттерны

3.1 Command - Команды для устройств

Реализуйте паттерн Command для управления устройствами с возможностью отмены.

Заготовка кода:

```

// Интерфейс команды
public interface Command {
    void execute();
    void undo();
    String getDescription();
}

// Команда включения устройства
public class TurnOnCommand implements Command {
    private Device device;

    public TurnOnCommand(Device device) {
        // TODO: Сохранить ссылку на устройство
    }
}

```

```

@Override
public void execute() {
    // TODO: Включить устройство
}

@Override
public void undo() {
    // TODO: Отменить (выключить устройство)
}

@Override
public String getDescription() {
    return "Включение " + device.getName();
}
}

// Команда выключения устройства
public class TurnOffCommand implements Command {
    private Device device;

    public TurnOffCommand(Device device) {
        // TODO: Сохранить ссылку на устройство
    }

    @Override
    public void execute() {
        // TODO: Выключить устройство
    }

    @Override
    public void undo() {
        // TODO: Отменить (включить устройство)
    }

    @Override
    public String getDescription() {
        return "Выключение " + device.getName();
    }
}

// Макрокоманда (выполняет несколько команд)
public class MacroCommand implements Command {
    private List<Command> commands;

    public MacroCommand() {
        this.commands = new ArrayList<>();
    }

    public void addCommand(Command command) {
        // TODO: Добавить команду в список
    }

    @Override

```

```

public void execute() {
    System.out.println("Выполнение макрокоманды...");
    // TODO: Выполнить все команды по порядку
}

@Override
public void undo() {
    System.out.println("Отмена макрокоманды...");
    // TODO: Отменить все команды в обратном порядке
}

@Override
public String getDescription() {
    return "Макрокоманда (" + commands.size() + " команд)";
}
}

// Пульт управления (Invoker)
public class RemoteControl {
    private Command lastCommand;
    private Stack<Command> commandHistory;

    public RemoteControl() {
        // TODO: Инициализировать историю команд
    }

    public void executeCommand(Command command) {
        // TODO:
        // 1. Выполнить команду
        // 2. Сохранить в lastCommand
        // 3. Добавить в историю
    }

    public void undoLastCommand() {
        // TODO: Отменить последнюю команду
    }

    public void showHistory() {
        System.out.println("=== История команд ===");
        // TODO: Вывести все команды из истории
    }
}

```

Задание: Создайте команду `SetTemperatureCommand` для термостата с возможностью отмены (возврат к предыдущей температуре).

3.2 Observer - Система уведомлений

Реализуйте паттерн Observer для уведомления пользователей об изменениях в доме.

Заготовка кода:

```
// Интерфейс наблюдателя
public interface SmartHomeObserver {
    void update(String event, String deviceName);
}

// Интерфейс субъекта
public interface SmartHomeSubject {
    void attach(SmartHomeObserver observer);
    void detach(SmartHomeObserver observer);
    void notifyObservers(String event, String deviceName);
}

// Система мониторинга (Subject)
public class MonitoringSystem implements SmartHomeSubject {
    private List<SmartHomeObserver> observers;

    public MonitoringSystem() {
        // TODO: Инициализировать список наблюдателей
    }

    @Override
    public void attach(SmartHomeObserver observer) {
        // TODO: Добавить наблюдателя
        System.out.println("Наблюдатель подключен к системе мониторинга");
    }

    @Override
    public void detach(SmartHomeObserver observer) {
        // TODO: Удалить наблюдателя
    }

    @Override
    public void notifyObservers(String event, String deviceName) {
        // TODO: Уведомить всех наблюдателей
    }

    // Метод для регистрации события
    public void registerDeviceEvent(String event, String deviceName) {
        System.out.println("[MONITOR] Событие: " + event + " для " + deviceName);
        notifyObservers(event, deviceName);
    }
}

// Уведомление на email
public class EmailNotifier implements SmartHomeObserver {
    private String email;

    public EmailNotifier(String email) {
        this.email = email;
    }

    @Override
```

```

        public void update(String event, String deviceName) {
            System.out.println("[EMAIL to " + email + "] Устройство " +
                               deviceName + ": " + event);
        }
    }

    // Уведомление в мобильное приложение
    public class MobileAppNotifier implements SmartHomeObserver {
        private String username;

        public MobileAppNotifier(String username) {
            this.username = username;
        }

        @Override
        public void update(String event, String deviceName) {
            System.out.println("[PUSH to " + username + "] " +
                               deviceName + ": " + event);
        }
    }

    // Логгер событий
    public class EventLogger implements SmartHomeObserver {
        @Override
        public void update(String event, String deviceName) {
            String timestamp = java.time.LocalDateTime.now().toString();
            System.out.println("[LOG " + timestamp + "] " +
                               deviceName + " - " + event);
        }
    }
}

```

Задание: Создайте наблюдатель `SecurityAlertObserver` , который:

- Отправляет тревожные уведомления
- Сохраняет критические события в отдельный файл (симулировать выводом в консоль)
- Считает количество тревожных событий

3.3 Strategy - Стратегии энергосбережения

Реализуйте паттерн Strategy для различных режимов работы умного дома.

Заготовка кода:

```

// Интерфейс стратегии
public interface EnergyStrategy {
    void applyStrategy(List<Device> devices);
    String getStrategyName();
}

// Стратегия максимальной производительности

```

```

public class PerformanceStrategy implements EnergyStrategy {
    @Override
    public void applyStrategy(List<Device> devices) {
        System.out.println("Применение режима максимальной производительности");
        // TODO: Включить все устройства
    }

    @Override
    public String getStrategyName() {
        return "Максимальная производительность";
    }
}

// Стратегия энергосбережения
public class EcoStrategy implements EnergyStrategy {
    @Override
    public void applyStrategy(List<Device> devices) {
        System.out.println("Применение режима энергосбережения");
        // TODO:
        // 1. Выключить все устройства кроме критичных
        // 2. Для термостатов установить температуру 18 градусов
    }

    @Override
    public String getStrategyName() {
        return "Энергосбережение";
    }
}

// Сбалансированная стратегия
public class BalancedStrategy implements EnergyStrategy {
    @Override
    public void applyStrategy(List<Device> devices) {
        System.out.println("Применение сбалансированного режима");
        // TODO:
        // 1. Включить 50% устройств
        // 2. Установить умеренную температуру
    }

    @Override
    public String getStrategyName() {
        return "Сбалансированный режим";
    }
}

// Контекст
public class EnergyManager {
    private EnergyStrategy strategy;
    private List<Device> devices;

    public EnergyManager(List<Device> devices) {
        this.devices = devices;
        // TODO: Установить стратегию по умолчанию
    }
}

```

```

    }

    public void setStrategy(EnergyStrategy strategy) {
        // TODO: Установить новую стратегию
        System.out.println("Стратегия изменена на: " + strategy.getStrategyName());
    }

    public void executeStrategy() {
        if (strategy != null) {
            strategy.applyStrategy(devices);
        }
    }
}

```

Часть 4: Интеграция всех паттернов

Создайте главный класс `SmartHomeDemo`, который демонстрирует работу всех реализованных паттернов.

Пример структуры:

```

public class SmartHomeDemo {
    public static void main(String[] args) {
        System.out.println("=== Инициализация системы умного дома ===\n");

        // 1. Singleton - Создание контроллера
        // TODO: Получить экземпляр SmartHomeController

        // 2. Builder - Создание конфигурации
        // TODO: Создать конфигурацию с помощью Builder

        // 3. Factory Method - Создание устройств
        // TODO: Создать различные устройства через фабрики

        // 4. Adapter - Интеграция старого устройства
        // TODO: Создать адаптер для старой лампы

        // 5. Decorator - Добавление функциональности
        // TODO: обернуть устройство в декораторы

        // 6. Observer - Система уведомлений
        // TODO: Создать систему мониторинга и наблюдателей

        // 7. Command - Управление устройствами
        // TODO: Создать команды и пульт управления

        // 8. Strategy - Режимы энергопотребления
        // TODO: Создать менеджер энергии и применить стратегии
    }
}

```

```

// 9. Facade - Упрощённые сценарии
// TODO: Использовать фасад для типичных сценариев

// 10. Демонстрация работы
demonstrateCommands();
demonstrateObservers();
demonstrateStrategies();
demonstrateFacade();
}

private static void demonstrateCommands() {
    System.out.println("\n=== Демонстрация паттерна Command ===");
    // TODO: Создать команды, выполнить, отменить
}

private static void demonstrateObservers() {
    System.out.println("\n=== Демонстрация паттерна Observer ===");
    // TODO: Зарегистрировать наблюдателей, создать события
}

private static void demonstrateStrategies() {
    System.out.println("\n=== Демонстрация паттерна Strategy ===");
    // TODO: Применить разные стратегии энергосбережения
}

private static void demonstrateFacade() {
    System.out.println("\n=== Демонстрация паттерна Facade ===");
    // TODO: Использовать упрощённые методы фасада
}
}

```

Примеры работы программы

Пример вывода:

```

=== Инициализация системы умного дома ===
Система умного дома инициализирована

```

Конфигурация:

Дом: Мой умный дом

Владелец: Иван Иванов

Комнат: 4

Система безопасности: Да

Климат-контроль: Да

```

=== Создание устройств ===

```

Устройство Лампа в гостиной создано и зарегистрировано

Устройство Термостат создано и зарегистрировано
Устройство Камера на входе создано и зарегистрировано

=== Демонстрация паттерна Decorator ===
[LOG] Включение устройства: Лампа в гостиной
[LOG] Время: 2024-01-15T10:30:00
[ENERGY] Режим энергосбережения активен для Лампа в гостиной
Лампа в гостиной включена

=== Демонстрация паттерна Command ===
Выполнение команды: Включение Термостат
История команд:
1. Включение Термостат
2. Выключение Камера на входе

Отмена последней команды...
Камера на входе включена

=== Демонстрация паттерна Observer ===
[MONITOR] Событие: Включено для Лампа в гостиной
[EMAIL to user@example.com] Устройство Лампа в гостиной: Включено
[PUSH to Иван] Лампа в гостиной: Включено
[LOG 2024-01-15T10:30:05] Лампа в гостиной - Включено

=== Демонстрация паттерна Strategy ===
Стратегия изменена на: Энергосбережение
Применение режима энергосбережения
Выключено устройств: 3
Установлена температура: 18°C

=== Демонстрация паттерна Facade ===
=== Запуск утреннего режима ===
Включены все лампы
Температура установлена: 22°C
Камеры выключены

Подсказки для выполнения

Для Singleton:

```
// Double-Checked Locking
if (instance == null) {
    synchronized (ClassName.class) {
        if (instance == null) {
            instance = new ClassName();
        }
    }
}
```

```
}  
return instance;
```

Для Factory Method:

```
// Фабрика создаёт конкретный объект  
public class ConcreteFactory extends AbstractFactory {  
    @Override  
    public Product createProduct(String name) {  
        return new ConcreteProduct(name);  
    }  
}
```

Для Builder:

```
// Цепочка вызовов (method chaining)  
public Builder setField(Type value) {  
    this.field = value;  
    return this; // Возвращаем this для цепочки  
}
```

Для Command:

```
// Stack для истории команд  
private Stack<Command> history = new Stack<>();  
  
public void executeCommand(Command cmd) {  
    cmd.execute();  
    history.push(cmd);  
}  
  
public void undo() {  
    if (!history.isEmpty()) {  
        Command cmd = history.pop();  
        cmd.undo();  
    }  
}
```

Для Observer:

```
// Уведомление всех наблюдателей  
public void notifyObservers(String event) {  
    for (Observer observer : observers) {  
        observer.update(event);  
    }  
}
```

```
}  
}
```

Справочная информация

Классификация паттернов GoF

Порождающие (Creational):

- **Singleton** - единственный экземпляр класса
- **Factory Method** - делегирование создания подклассам
- **Abstract Factory** - семейства связанных объектов
- **Builder** - пошаговое создание сложных объектов
- **Prototype** - клонирование объектов

Структурные (Structural):

- **Adapter** - преобразование интерфейсов
- **Bridge** - отделение абстракции от реализации
- **Composite** - древовидные структуры
- **Decorator** - динамическое добавление функциональности
- **Facade** - упрощённый интерфейс к подсистеме
- **Flyweight** - разделение ресурсов
- **Proxy** - контроль доступа к объекту

Поведенческие (Behavioral):

- **Chain of Responsibility** - цепочка обработчиков
- **Command** - инкапсуляция запросов
- **Iterator** - последовательный доступ
- **Mediator** - координация взаимодействия
- **Memento** - сохранение состояния
- **Observer** - подписка на события
- **State** - изменение поведения при смене состояния
- **Strategy** - взаимозаменяемые алгоритмы
- **Template Method** - скелет алгоритма
- **Visitor** - новые операции без изменения классов
- **Interpreter** - интерпретация языка

Когда использовать какой паттерн

Используйте Singleton когда:

- Нужен единственный экземпляр класса (конфигурация, логгер, пул соединений)
- Требуется глобальная точка доступа

Используйте Factory Method когда:

- Класс заранее не знает, объекты каких классов нужно создавать
- Класс хочет делегировать создание объектов подклассам

Используйте Builder когда:

- У объекта много параметров, многие из которых опциональны
- Нужен контроль над процессом конструирования

Используйте Adapter когда:

- Нужно использовать существующий класс с несовместимым интерфейсом
- Интеграция старого кода в новую систему

Используйте Decorator когда:

- Нужно динамически добавлять функциональность объектам
- Расширение через наследование неудобно

Используйте Facade когда:

- Нужно предоставить простой интерфейс к сложной подсистеме
- Хотите уменьшить зависимости между подсистемами

Используйте Command когда:

- Нужно параметризовать объекты выполняемым действием
- Требуется поддержка отмены операций
- Нужна очередь запросов или логирование

Используйте Observer когда:

- Изменение одного объекта требует изменения других
- Нужна система подписки на события
- Количество зависимых объектов может меняться

Используйте Strategy когда:

- Нужны различные варианты алгоритма

- Множество похожих классов отличаются только поведением
- Нужно изолировать бизнес-логику от реализации алгоритмов

Частые ошибки

1. Singleton

Неправильно:

```
public static SmartHomeController getInstance() {  
    if (instance == null) {  
        instance = new SmartHomeController(); // Не потокобезопасно!  
    }  
    return instance;  
}
```

Правильно:

```
public static SmartHomeController getInstance() {  
    if (instance == null) {  
        synchronized (SmartHomeController.class) {  
            if (instance == null) {  
                instance = new SmartHomeController();  
            }  
        }  
    }  
    return instance;  
}
```

2. Builder

Неправильно:

```
public Builder setField(int value) {  
    this.field = value;  
    // Забыли вернуть this!  
}
```

Правильно:

```
public Builder setField(int value) {  
    this.field = value;  
    return this;  
}
```

```
    return this; // Для цепочки вызовов
}
```

3. Decorator

Неправильно:

```
public void turnOn() {
    // Добавили свою логику, но забыли вызвать decoratedDevice
    System.out.println("Логирование");
}
```

Правильно:

```
public void turnOn() {
    System.out.println("Логирование");
    decoratedDevice.turnOn(); // Делегируем вызов!
}
```

4. Command

Неправильно:

```
public void undo() {
    device.turnOn(); // Просто включаем, не помним предыдущее состояние
}
```

Правильно:

```
public void undo() {
    // Отмена = противоположное действие
    if (wasOn) {
        device.turnOn();
    } else {
        device.turnOff();
    }
}
```

5. Observer

Неправильно:

```
public void notifyObservers() {
    for (Observer obs : observers) {
```

```
        obs.update(); // Забыли передать данные о событии
    }
}
```

Правильно:

```
public void notifyObservers(String event, String device) {
    for (Observer obs : observers) {
        obs.update(event, device); // Передаём информацию
    }
}
```